

SAHTY BACKEND

Code Review Report

Generated: May 16, 2026

Overall Rating	Score
Backend Quality	4.5/10 ⚠ CRITICAL ISSUES

EXECUTIVE SUMMARY

This healthcare platform backend has critical security vulnerabilities, architectural issues, and zero test coverage that make it unsuitable for production deployment. The codebase exhibits poor error handling, missing input validation, and exposes sensitive data through committed secrets files. Major issues include hardcoded credentials in git, unauthenticated access to medical data, and logic errors in alert scheduling. Immediate remediation is required before any user data can be safely stored.

SECURITY ISSUES (CRITICAL)

1. Hardcoded Secrets in Version Control

.env file committed to git exposing email credentials, database URL, API keys, and SECRET_KEY

2. Overly Permissive CORS

CORS_ALLOW_ALL_ORIGINS = True allows requests from any origin, contradicting specific origins

3. Authentication Bypass - AllowAny Permissions

Multiple endpoints (medical_files, measurements) use AllowAny, exposing sensitive medical data

4. DEBUG Mode Enabled in Production

DEBUG=True in .env exposes stack traces and sensitive information to users

5. Firebase Credentials Committed

firebase_credentials.json committed to git allows Firebase project hijacking

6. Debug Print Statements

Leftover print() calls leak sensitive data to logs (firebase.py:18, alerts/services.py:100, medical_files/views.py:80)

7. Brute Force - Missing OTP Rate Limiting

No rate limiting on OTP verification attempts, only on login (5 attempts)

8. Token Bloat - Excessive JWT Claims

JWTs embed patient_id/doctor_id unnecessarily, increasing token size and data exposure

9. Soft Delete Enables Account Resurrection

Deleted accounts can be re-created with same email by appending '.deleted' instead of permanent deletion

10. No File Upload Validation

Medical file uploads lack type validation, size limits, and malware scanning

BUGS

1. Logic Error in Alert Reminder (CRITICAL)

alerts/services.py:74 - break statement exits outer loop instead of inner, skipping remaining treatments

2. Race Condition in User Registration

Check-then-create pattern allows concurrent registration with same email/phone, creating duplicates

3. Undefined Variable Reference

medical_files/services.py:54-55 - access_exists may be None when referenced

4. Missing Null Checks on Measurements

measurements/services.py:26,28 - .latest() throws DoesNotExist if patient has no measurements

5. Typo in Method Name

measurements/views.py:30 - getLeastestMeasurements should be getLatestMeasurements

6. Unused Import

medical_files/services.py:4 - drf_spectacular.types.date never used

7. Incomplete Email Verification Logic

users/services.py:20-24 - User created before email verified, timing allows unverified accounts

SOLID PRINCIPLE VIOLATIONS

1. Single Responsibility Principle (SRP)

`AlertService.createMedicationReminder()` handles queries, email, push notifications, and DB updates

2. Open/Closed Principle

New alert types require new methods; should use polymorphic factory pattern

3. Dependency Inversion

Views depend directly on concrete service implementations instead of abstractions

4. Interface Segregation

All views inherit same `permission_classes=[AllowAny]` instead of specific mixins

DEAD CODE & UNUSED DEPENDENCIES

Commented Out Code: config/settings.py:118-122, config/urls.py:21,38-39, measurements/services.py:59-73, medical_files/services.py:15, alerts/services.py:17-23. Unused Imports: medical_files/services.py:4, medical_files/models.py:4. Dead Apps: notifications app exists but disabled, Django Channels configured but unused.

BAD PRACTICES & POOR STRUCTURE

1. Weak Naming Conventions

value1, value2 in Measurement (generic), tmpPostgres (poor naming)

2. Empty Response Objects

Views return raw dictionaries instead of proper Response objects with consistent format

3. Missing ORM Best Practices

.latest() without exists check, N+1 queries, missing .select_related()

4. Hardcoded Magic Numbers

5 failed attempts, 15 min lockout, 7 days, 14 days - should be settings constants

5. Inconsistent Error Handling

Generic except Exception blocks, inconsistent HTTP status codes for same errors

6. Implicit Type Conversions

measurements/services.py:40 - float conversions without validation

7. Missing Pagination

Large datasets like measurements lack pagination

MISSING ERROR HANDLING & VALIDATION

No Input Validation: Patient height/weight lack max values, phone lacks format validation. Missing Transaction Safety: Most operations lack atomic(), partial failures leave inconsistent state. No Request Validation: Medical files not validated for requester ownership. No Audit Logging: No access records or timestamps. Missing 2FA Validation: Enables email enumeration attacks.

MISSING TEST COVERAGE (CRITICAL)

Status: Only stub tests.py files exist (3 lines each). Coverage: 0% of codebase tested. Missing: Authentication flows, OTP brute force, permission checks, measurement risk assessment, medication scheduling, medical file access control, race conditions, error handling. This is absolutely critical for healthcare applications.

PERFORMANCE ISSUES

1. N+1 Queries in Alert Services

alerts/services.py:87-108 - Loop over appointments with nested user queries

2. Missing Query Optimization

Inconsistent use of `select_related()`, missing `prefetch_related()`

3. Large Token Payload

JWT tokens 1-2KB with embedded `patient_id/doctor_id/name`

4. Unoptimized Appointment Query

alerts/services.py:91 - `filter().first()` inefficient for existence check

MISSING FEATURES & IMPROVEMENTS

No API Versioning: Breaking changes affect clients. No Rate Limiting: DoS vulnerability. No Request Logging: Can't debug or detect abuse. No Data Encryption: Plain FileField storage. Missing Database Indexes: N+1 queries. No Soft Delete: No audit trail. No Backup Strategy: Data loss risk. Incomplete Documentation: OpenAPI schema lacks descriptions.

CODE QUALITY SUMMARY

Category	Rating	Notes
Security	2/10	Critical secrets leaks, no auth on sensitive endpoints
Error Handling	3/10	Generic exceptions, missing validation
Architecture	3/10	Mixed concerns, poor separation
Performance	4/10	N+1 queries, unoptimized
Testing	0/10	Zero test coverage
Documentation	3/10	No docstrings, magic numbers
Best Practices	3/10	Django ORM misuse, weak patterns

PRIORITY RECOMMENDATIONS

CRITICAL: Remove secrets from git, fix AllowAny permissions, add test suite, fix race conditions, fix alert logic, set DEBUG=False, remove debug prints. HIGH PRIORITY: Add input validation, implement error handling, fix N+1 queries, add transactions, add audit logging, fix file uploads. MEDIUM: Improve architecture, remove dead code, add logging, optimize queries, add versioning, add docstrings.

CONCLUSION

This healthcare application backend has critical security vulnerabilities and code quality issues unsuitable for production. Hardcoded secrets, unauthenticated medical data access, and zero test coverage are immediate blockers. The codebase requires: comprehensive security audit, complete test coverage, code review process, architectural improvements (SOLID), input validation, and proper access control. Do not deploy until all critical and high-priority issues are resolved.